CENG-567
REINFORCEMENT LEARNING

Lecture-10
Polic-based methods

Dr. Osman GÖKALP

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap (From Tabular RL to Deep RL)

Why did we move beyond Dynamic Programming and tabular methods?

DP limitations

Requires full knowledge of environment dynamics

Uses tabular representations → does not scale

MC & TD

Learn from interaction (no model required)

Still rely on tables → scalability problem remains

Key challenge

Real-world RL has **huge or continuous state spaces**

Example: raw images → impossible to enumerate states

Solution

Use **function approximation**, especially **neural networks**, to represent value functions

Recap (Deep Q-Learning and Stability Challenges)

Replacing tables with neural networks is powerful—but dangerous

Neural-Fitted Q-Iteration (NFQ)

- Batch learning
- Repeatedly fit Q-network to bootstrapped targets
- Computationally expensive (retrain from scratch)

Online Q-Learning

- Incremental gradient updates after each transition
- More scalable, but:
  - Noisy gradients
  - Moving targets
  - Correlated samples

The Deadly Triad

- Function approximation
- Bootstrapping
- Off-policy learning

→ **theoretical instability & possible divergence**

Recap (DQN and the Road to Rainbow)

How Deep Q-Learning became practical

DQN key stabilizing ideas

- **Experience Replay** → breaks correlations, improves data efficiency
- **Target Network** → stabilizes bootstrapping targets

Rainbow DQN

Combines multiple improvements:

- Double Q-learning (reduces overestimation)
- Prioritized Replay (focus on informative samples)
- Dueling Networks (separate state value & advantage)
- Multi-step returns (less bootstrapping)
- Distributional RL (learn return distributions)
- Noisy Nets (learned exploration)

Main takeaway

Deep Q-learning works **not because theory guarantees it**,
but because **carefully engineered stabilizing tricks** make it work in practice

Policy-Based Methods — Motivation

Limitations of Value-Based Methods

- Perform well in **discrete action spaces**
- Struggle with **continuous control** (e.g., robotics)

What Are Policy-Based Methods?

- Directly **parameterize and optimize a policy**
- Naturally handle **continuous action spaces**

Key Advantages

- Stronger **theoretical foundations** (direct policy optimization)
- Can learn **stochastic policies**
  - Essential for **partial observability**
  - Important in **games and exploration**

Big Picture

- Policy-based methods **complement** value-based approaches
- Expand RL to a broader class of real-world problems

Why Policy-Based Methods?

Value-Based (Q-learning)

Learns **action values** first
Policy is derived **indirectly** via arg max over Q-values

Key Question

Do we really need accurate **action values**?
Often, **no** — they are only a means to obtain a good policy

Policy-Based Methods

Learn a **policy directly**: $\pi(a \mid s; \theta)$
No intermediate action-value estimation

Why This Is More Principled?

Optimize the **policy itself**, not a proxy
Use **gradient ascent directly on expected return**

Practical Advantage

More efficient when:

- Many actions have **similar or poor values**
- Precise Q-values are unnecessary

Continuous Action Spaces: Why Value-Based Methods Struggle?

Value-based methods (DP, MC, TD, Q-learning)

- Designed for **discrete action spaces**

Many real-world problems need continuous actions

- Robotics (torque, velocity, angle)
- Control systems

Discretizing actions

- Leads to **unnatural and poor behavior**
- Action resolution becomes a bottleneck

Why Continuous Actions Are Hard for Q-Learning?

- Q-learning relies on **action maximization**:

$$y = r + \gamma \max_{a'} Q(s', a')$$

$$a^* = \arg\max_{a} Q(s, a)$$

In **continuous action spaces**:

Maximization is non-trivial

Finding the best action requires solving a continuous optimization problem rather than selecting from a finite set.

No closed-form solution in general

Because the Q-function is a nonlinear neural network, its maximum with respect to actions cannot be computed analytically.

Brute-force search is impractical

Accurate maximization would require sampling an enormous number of actions, which is computationally infeasible in high-dimensional spaces.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Workarounds for Continuous Actions in Value-Based RL

Sampling-Based Maximization

Sample actions from the continuous space
Choose the best sampled action
Or fit a surrogate and maximize it (cross-entropy method - CEM)
❌ Limited precision, computationally expensive

Restricted Function Classes

Use Q-functions quadratic in actions
Enables analytical maximization
Example: Normalized Advantage Functions - **NAF (Gu et al., 2016)**
❌ Less expressive than deep networks

Separate Maximization Network

Learn another function to approximate **arg max**
Example: Deep Deterministic Policy Gradient - **DDPG**
✅ Competitive, but more complex

Why Policy-Based Methods Are Preferred?

Directly represent a **continuous-action policy**

No action maximization required

More expressive and scalable

Cleaner optimization via **policy gradients**

Key Takeaway

Continuous actions break the core assumption of Q-learning

Value-based fixes exist, but are **limited or complex**

Policy-based methods naturally handle continuous control

Important Note

Most policy-based methods support both discrete and continuous action spaces

The ability to learn truly random stochastic policies

- Throughout Q-learning, we used **soft-policies** such as ϵ **-greedy** to enable the agent to **explore** the environment during **training**.
- Although this approach works pretty well in practice, and it can be made more sophisticated by **annealing** the ϵ, it is still **not** a **learned** parameter.
- Policy-based methods can **learn random policies** that lead to a **more principled exploration during training**.
- Perhaps a bigger issue is that we may need to learn random policies not just for training **but also for inference**. There are two **reasons** why we might want to do this.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Inference in POMDPs

- In **partially observable environments** (POMDPs), we may have what are called **aliased states**, which **emit the same observation** although the **states themselves are different**, for which the **optimal actions might be different**.
- The agent only observes the shapes of the states it is in but cannot tell what the state is.
- The **optimal action** when the agent observes a hexagon is a **random one** because a deterministic policy would make the agent get stuck either between 1 and 2.

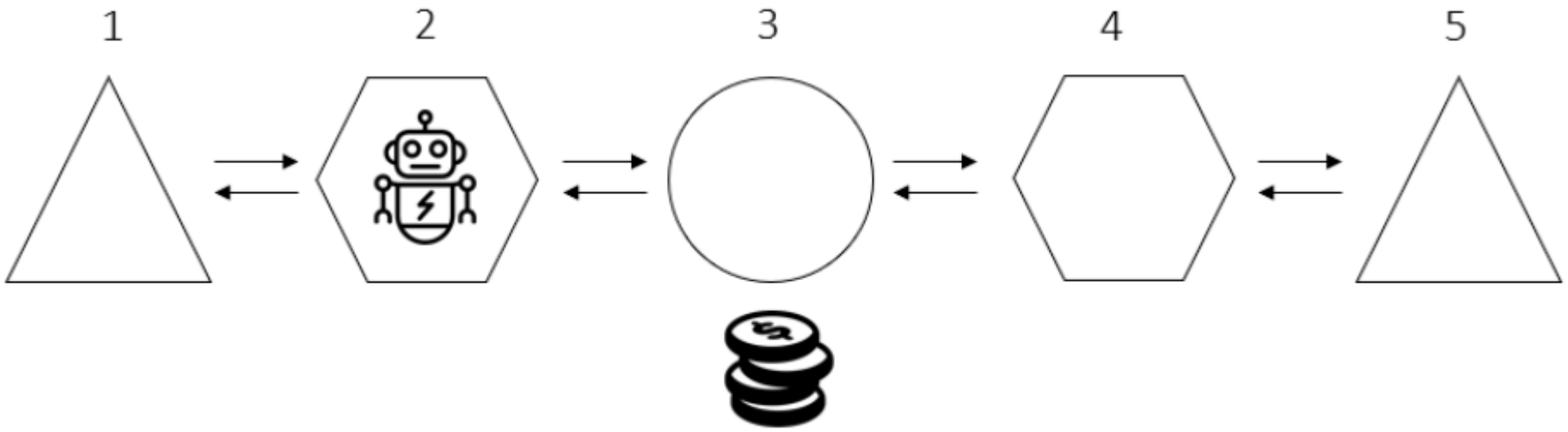
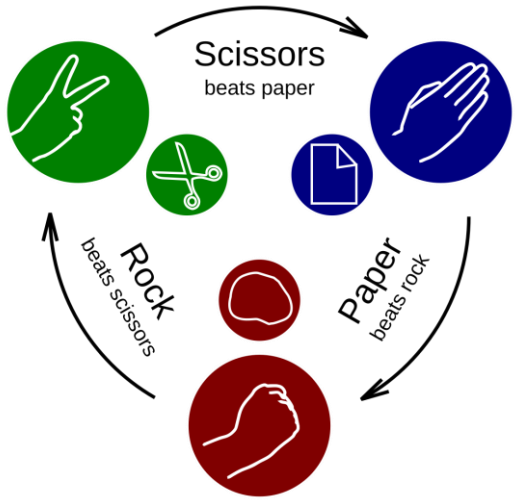


Figure 7.1 – Robot in a partially observable environment

Inference with adversarial agents

- In game settings with **adversarial agents**, there could be **cases** where the only **optimal policy** is a **random** one. The canonical example for this is rock-paper-scissors, where the optimal policy is to select an action uniformly at random. **Any other policy could be exploited by the opponents in the environment**.
- **Value-based methods don't have the ability** to learn such random policies for inference whereas policy-based methods allow that.
- **Tip**: If the **environment** is **fully observable**, and it is **not a game setting**, **there is always a deterministic policy that is optimal** (although there could be **more than one optimal policy and some of them could be random**). In such cases, we don't need a random policy during inference.



Policy gradient: Objective Function

- Policy gradient methods, **directly focus on optimizing the policy** with respect to the reinforcement learning objective (expected discounted return):

$$J(\pi_\theta) = E_{\tau \sim \rho_\theta(\tau)} \left[\sum_{t=0}^{T} \gamma^t R(S_t, A_t) \right]$$

- $J(\pi_\theta)$
  - The objective we aim to **maximize**. Depends on the policy π_θ
- **Policy parameters** θ
  - Define the policy behavior. Learned during training
- **Trajectory** $\tau \sim \rho_\theta(\tau)$
  - Sequence of states and actions generated (sampled) by the policy. Distribution depends on π_θ ,hence on θ
- **Reward function** $R(S_t, A_t)$
  - Provided by the environment. Unknown to the agent. Depends on state–action pairs

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Policy gradient: Update parameters with gradient ascent

- we have an objective function J that we want to **maximize**, which **depends on parameter** θ, which we can control. A natural thing to do is to take a gradient step in the ascending direction:

$$\theta_{k+1} = \theta_k + \alpha \, \nabla_\theta J\left(\pi_{\theta_k}\right)$$

- α is the step size.
- Updating θ means that we **directly optimize** the policy.

- We can estimate the gradient from a batch of N trajectories as follows (refer the textbook for the derivation):
  Policy gradients push the policy to repeat actions that worked well and avoid those that didn't.

$$\nabla_\theta J(\pi_\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta \left(a_t^{(i)} \mid s_t^{(i)} \right) \right) \left(\sum_{t=0}^{\infty} \gamma^t \, R\left(s_t^{(i)}, a_t^{(i)} \right) \right)$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Policy Gradient Estimate — Intuition

- **Average over episodes (N)**
  - The gradient is estimated using **multiple sampled trajectories**. We take the gradient with respect to θ of the log-probability of the selected action.

$$\nabla_\theta \log \pi_\theta \left(a_t \mid s_t\right)$$

  - Measures how changing θ affects the probability of the taken action
  - Tells us **which direction to change the policy**
- **Return term** $\sum_t \gamma^t R(s_t, a_t)$
  - Weights the gradient by how good the trajectory was
  - **Good outcomes are reinforced**, bad ones are discouraged
- **Key idea**
  - Increase the probability of actions that led to **high return**
  - Decrease the probability of actions that led to **low return**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

REINFORCE algorithm

- One of the **earliest** policy gradient methods
- **Needs** a lot of **improvements** on top of this to come up with methods with which we can attack realistic problems
- On the other hand, **understanding** REINFORCE is **useful** to formalize these ideas in the context of an algorithm.
- The fact that the sampled trajectories are used to obtain a gradient estimate for the **policy parameters at hand** makes policy gradient methods **on-policy**. Therefore, we **cannot use samples obtained under a different policy** to improve the existing policy, unlike in value-based methods.
- The REINFORCE algorithm **requires complete trajectories** for network updates, therefore it is a **Monte Carlo** method.

1. Initialize a policy π_θ:

   while some stopping criterion is not met do:

2. Collect N trajectories $\{\tau^i\}$ from the environment using π_θ.
3. Calculate

$$\nabla_\theta J(\pi_\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t^i|s_t^i)\right)\left(\sum_{t=0}^{T}R(s_t^i, a_t^i)\right).$$

4. Update $\theta \coloneqq \theta + \alpha\nabla_\theta J(\pi_\theta)$:

   end while

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The problem with REINFORCE and all policy gradient methods

Main Issue

- The most important limitation of **REINFORCE** and policy gradient methods is the **high variance of the gradient estimate** $\nabla_\theta J(\pi_\theta)$.

This high variance can destabilize learning and slow convergence.

Why does the gradient have high variance?

- **Stochastic environments**
  Even with the same policy, environment randomness can generate very different trajectories, leading to highly variable gradient estimates.
- **Variable trajectory lengths**
  Episodes may terminate at different times, producing gradients that are sums over very different numbers of terms.
- **Sparse rewards**
  When rewards are rare, most trajectories provide little learning signal, while a few dominate the gradient estimate.
- **Limited number of samples (N)**
  In practice, N is kept small for efficiency, which may be insufficient to accurately capture the full trajectory distribution.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Replacing Total Reward with Reward-to-Go

Original REINFORCE Gradient

$$\nabla_\theta J(\pi_\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta\left(a_t^i \mid s_t^i\right)\left(\sum_{t'=0}^{T} R\left(s_{t'}^i, a_{t'}^i\right)\right)$$

Issue:
Each policy gradient term is weighted by the **total reward of the entire trajectory**, regardless of when the action was taken.

Why is this problematic?

- Actions at time t **cannot influence past rewards**
- Early rewards add **unnecessary noise**
- Large reward sums → **high variance updates**

This violates the intuition of **causality**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Reward-to-Go Idea (Causal Policy Gradient)

- Weight each log-probability term only by **future rewards**:

$$\nabla_\theta J(\pi_\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta\left(a_t^i \mid s_t^i\right)\left(\sum_{t'=t}^{T} R\left(s_{t'}^i, a_{t'}^i\right)\right)$$

Key Properties

✓ **Unbiased gradient estimate**
✓ **Lower variance** (smaller reward sums)
✓ Respects **causality**
✓ More stable learning

Interpretation

$$\sum_{t'=t}^{T} R\left(s_{t'}^i, a_{t'}^i\right) \approx Q^\pi\left(s_t^i, a_t^i\right)$$

→ Reward-to-go is an **estimate of the action-value function**.

- **This improvement** over the REINFORCE algorithm is a form of a ***vanilla policy gradient*** method.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Vanilla policy gradient

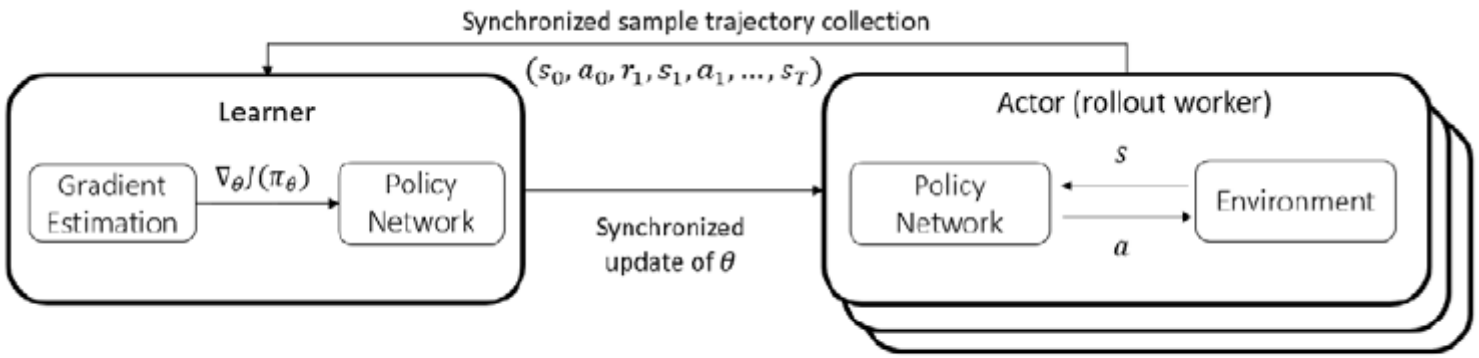


Figure 7.2 – Vanilla policy gradient architecture

- Can easily be parallelized with multiple rollout workers (actors).
- Since policy gradient methods are **on-policy**, we need to make sure that the **samples** we use to update the neural network parameters (weights) **come from the existing policy** suggested by the network.
- This **dictates synchronizing** the policy in use across the rollout workers.

Actor-critic methods

Overview

• Actor–critic methods are designed to address the **high variance** problem in policy gradient algorithms
• They extend ideas from classic policy gradient methods such as REINFORCE
• When combined with deep neural networks, they can solve **complex, real-world RL tasks**

Why Do We Need Actor–Critic?

• Policy gradient estimates can be very **noisy**
• Earlier, variance was reduced using **reward-to-go** instead of full trajectory return
• Although helpful, reward-to-go alone often **does not sufficiently reduce variance**

Actor-critic methods

Reward-to-Go as a Value Estimate

The reward-to-go:

$$\sum_{t'=t}^{T} R\left(s_{t'}^{i}, a_{t'}^{i}\right)$$

- **Acts as** an estimate of the **action-value function** $Q^{\pi}(s_{t'}, a_t)$
- This **estimate depends** on the **current policy** π_θ
- Different trajectories visiting the same (s, a)pair produce **different estimates**

➞ This variability contributes significantly to **gradient variance**

Actor-critic methods

Important Clarification

- **Policy gradient** methods estimate:

$$Q^{\pi}(s, a)$$

→value under the **current (behavior) policy**

- **Q-learning** methods estimate:

$$Q(s, a)$$

→value under the **greedy optimal policy (using target policy with arg max operator)**

These two quantities are conceptually different.

Actor-critic methods

Reducing Variance Further: Key Idea

- Instead of relying on many noisy reward-to-go samples
- Learn a **function approximator** (neural network) for $Q^{\pi}(s, a)$
- Train it using sampled reward-to-go values as targets
- For each (s, a) ,the network outputs **one consistent estimate**

→ This greatly reduces variance in policy updates

Actor–Critic Interpretation

- The **policy network** decides **which action to take** → *Actor*
- The **value network** evaluates **how good the action is** → *Critic*

The **critic** assesses the policy, while the **actor** improves it.

Incremental Critic Training

- The critic is **not retrained from scratch** after each policy update
- It is updated gradually using new experience
- As a result, the critic reflects **older versions of the policy**

→ This **introduces bias, but the reduced variance** is usually **worth** it

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Actor-critic methods

Baselines (motivation)

- Policy gradient updates depend heavily on the **scale of rewards**
- Equivalent tasks with different reward magnitudes can lead to:
  - Very large gradients
  - Very small gradients
- This makes learning unstable and inefficient

Intuition Behind Baselines

We want to know:

Did this trajectory perform **better or worse than expected**?

Not simply whether the reward was large in absolute terms.

Actor-critic methods

Subtracting a Baseline from the reward sum

- Replace total return with:

$$R(\tau) - b$$

- The gradient estimate remains **unbiased**
- Variance is significantly reduced

A simple and effective baseline is the "average trajectory return":

$$b = \frac{1}{\mathrm{N}} \sum_{\mathrm{i}=1}^{\mathrm{N}} \mathrm{R}(\tau^i)$$

Baseline + Reward-to-Go

When combined with using reward-to-go estimates, a natural choice for the baseline is the state value, which results in the following gradient estimate:

$$\nabla_\theta J(\pi_\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta \left(a_t^{(i)} \mid s_t^{(i)} \right) A^\pi \left(s_t^{(i)}, a_t^{(i)} \right)$$

$$A^\pi \left(s_t^{(i)}, a_t^{(i)} \right) = Q^\pi \left(s_t^{(i)}, a_t^{(i)} \right) - V^\pi \left(s_t^{(i)} \right)$$

Actor-critic methods

Meaning of the Advantage

- Measures how much better an action is **compared** to the **policy's average behavior**
- **Positive** advantage → action is better than expected
- **Negative** advantage → action is worse than expected

This provides a **relative performance signal**

Advantage Actor–Critic

- The critic estimates:
  – $V(s)$,$Q(s, a)$,or directly $A(s, a)$
- The actor updates the policy using the advantage
- Combines low variance with effective learning

→ Forms the basis of **advantage actor–critic algorithms**

Key Takeaway

- Actor–critic methods reduce variance by:
  – Learning value estimates
  – Using baselines
- They accept mild bias in exchange for **stable and efficient learning**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Advantage Actor-Critic – A2C

Advantage Estimation in A2C

To complete the A2C algorithm, we need a practical way to compute the **advantage**.

In principle, a critic could:

- Directly approximate the advantage $A^{\pi}(s, a)$, or
- Learn the action-value $Q^{\pi}(s, a)$ and derive the advantage from it.

Both approaches require a network that depends on **state and action**.

Key simplification

Instead of learning $A^{\pi}(s, a)$ or $Q^{\pi}(s, a)$ directly, A2C relies on a **state-value function**.

This leads to a simpler critic that depends **only on the state**, while still enabling advantage estimation.

Advantage Actor-Critic – A2C

Computing Advantage Using a Value Critic

Single-step transition information

From interaction with the environment, we observe tuples:

$$(s_t, a_t, r_{t+1}, s_{t+1})$$

The immediate reward and next state allow us to form a **bootstrapped estimate** of the action-value:

$$Q^{\pi}(s_t, a_t) \approx r_{t+1} + \gamma V^{\pi}(s_{t+1})$$

Advantage estimation

The advantage is then computed as:

$$A^{\pi}(s_t, a_t) = r_{t+1} + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$$

- This approach allows us to use a **neural network** that **simply estimates the state values** to obtain an advantage estimate.

Interpretation

The **critic evaluates** states **via** $\boldsymbol{V^{\pi}(s)}$

The **advantage** measures whether the chosen action performed better or worse than expected

To train the critic, we use bootstrapping to compute targets for the state-value function; given a sampled transition $(s_t, a_t, r_{t+1}, s_{t+1})$, the target for $V^{\pi}(s_t)$ is $r_{t+1} + \gamma V^{\pi}(s_{t+1})$, which also serves as a one-step estimate of $Q^{\pi}(s_t, a_t)$ since actions are drawn from the current stochastic policy.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

A2C architecture

Synchronized data collection

A2C uses **multiple rollout workers** that operate **in parallel**.

At any given time, **all workers share the same policy parameters**.

Each worker collects samples using this common policy.

The collected samples are sent to a **learner**, which updates:

- the **actor** (policy network)
- the **critic** (value network)

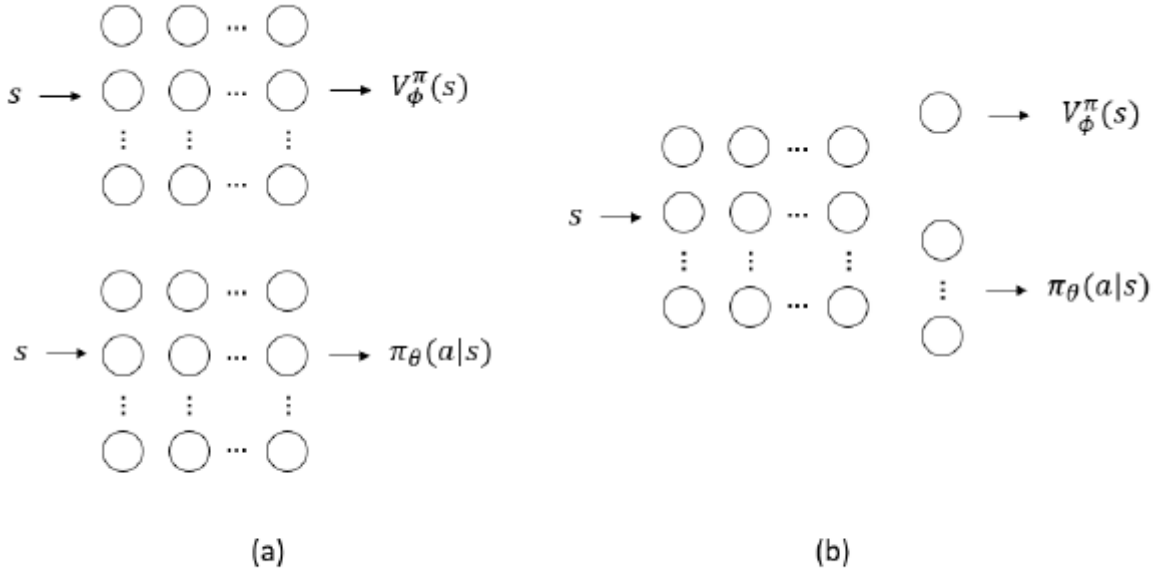


Figure 7.4 – Isolated versus shared neural networks

Relation to vanilla policy gradient

The overall architecture closely resembles vanilla policy gradient methods.

The key difference is the introduction of a **critic**, which provides value estimates to guide learning.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

A2C architecture

Trade-offs

- Separate networks:
  - Easier to tune
  - Less risk of instability

- Shared networks:
  - More parameter-efficient
  - Beneficial when feature extraction is costly (e.g., image-based inputs)
  - Requires careful hyperparameter tuning, especially learning rates

Design flexibility

Architectures can lie anywhere between fully shared and fully separate, depending on stability and efficiency needs.

The A2C algorithm

1. Initialize the actor and critic network(s), θ and ϕ.

   while some stopping criterion is not met do:

2. Collect a batch of N samples $\{(s_i, a_i, r_i, s_i')\}_{i=1}^{N}$ from the (parallel) environment(s) using π_θ.
3. Obtain the state-value targets $\left\{y_i \middle| y_i = r_i + \gamma V_\phi^\pi(s_i')\right\}_{i=1}^{N}$.
4. Use gradient descent to update ϕ with respect to a loss function $L(V_\phi^\pi(s_i), y_i)$, such as squared loss.
5. Obtain the advantage value estimates:
   $\left\{A^\pi(s_i, a_i) \middle| A(s_i, a_i) = r_i + \gamma V_\phi^\pi(s_i') - V_\phi^\pi(s_i)\right\}_{i=1}^{N}$.
6. Calculate
   $$\nabla_\theta J(\pi_\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log \pi_\theta(a_i|s_i)\, A^\pi(s_i, a_i).$$
7. Update
   $\theta := \theta + \alpha \nabla_\theta J(\pi_\theta)$.
8. Broadcast the new θ to the rollout workers.

   end while

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Off-policy methods

- **Policy-based methods** are typically **on-policy**
  - Require collecting **new samples after every policy update**
  - Can be **very expensive** when environment interaction is costly
- **Value-based methods** are **off-policy**
  - Can **reuse past experience** efficiently
  - But are usually limited to **discrete action spaces**
- **Key gap**
  - Need methods that are:
    - **Off-policy** (sample efficient)
    - **Compatible with continuous action spaces**

➡️ This motivates **off-policy actor–critic methods** (e.g., DDPG, TD3, SAC).

DDPG – Deep Deterministic Policy Gradient

DDPG as an Extension of Deep Q-Learning

- **Deep Q-learning (DQN)** learns an action-value function:
  - Q(s,a)
  - Selects actions via: $a^* = \arg\max_a Q(s,a)$
- **Problem with continuous action spaces**
  - Learning Q(s,a) is feasible
  - But computing **arg max** over **continuous actions** is **computationally intractable**
- **How DDPG solves this?**
  - Introduces a **deterministic actor network** that directly outputs the best action
  - Uses a **critic network** to estimate Q(s,a)
  - Avoids explicit maximization over the action space

➡️ DDPG enables **off-policy learning in continuous action spaces**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

How DDPG handles continuous action spaces

DDPG simply learns another approximation, $\mu_\phi(s)$, that estimates the best action given the state. If you wonder why this works, consider the following thought process:

- DDPG assumes that the continuous action space is differentiable with respect to a.
- For a moment, also assume that the action values, $Q(s,a)$, are known or have been learned.
- Then the problem simply becomes learning a function approximation, $\mu_\phi(s)$, whose input is s, output is a, and the parameters are ϕ. The "reward" for the optimization procedure is simply provided by $Q(s,a)$.
- We can therefore use a gradient ascent method to optimize ϕ.
- Over time, the learned action values will hopefully converge and be less of a moving target for the policy function training.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

DDPG

- Because DDPG assumes the **differentiability** of the policy function with respect to action, **it can only be used with continuous action spaces**.
- DDPG is an e**xtension of deep Q-learning**, plus learning a **policy function**.
- Many approaches such as prioritized experience replay and multi-step learning, can be used to form a DDPG variate. The original DDPG algorithm, on the other hand, is closer to the DQN algorithm and uses the following:

  - A **replay buffer** to store experience tuples, from which the sampling is done uniformly at random.
  - A **target network**, which is updated using Polyak averaging as in $\theta_{target} := \rho\theta_{target} + (1-\rho)\theta$, rather than syncing it with the behavior network every C steps.

DDPG

DDPG then replaces the target calculation in the DQN algorithm:

$$y = r + \gamma \max_{a'} Q_{\theta_{target}}(s', a')$$

with this one:

$$y = r + \gamma Q_{\theta_{target}}\left(s', \mu_{\phi_{target}}(s')\right)$$

Another important difference between DQN and DDPG is that DQN uses ϵ-greedy actions during training. The policy network in DDPG, however, provides a deterministic action for a given state, hence the word "deterministic" in the name. To enable exploration during training, some noise is added to the action. More formally, the action is obtained as follows:

$$a = clip(\mu_{\phi_{target}}(s) + \epsilon, a_{Low}, a_{High})$$

where ϵ can be chosen as white noise (although the original implementation uses what is called OU noise). In this operation, a_{Low} and a_{High} represent the boundaries of the continuous action space.

DDPG's shortcomings

Despite its initial popularity, there are several issues that the algorithm runs into:

- It can be quite sensitive to hyperparameter selections.
- It runs into the issue of maximization bias while learning action values.
- Spikes in the action-value estimates (which are potentially erroneous) are exploited by the policy network and derail the learning.

Other off-policy deep RL methods:

TD3 – Twin Delayed Deep Deterministic Policy Gradient

SAC – Soft actor-critic